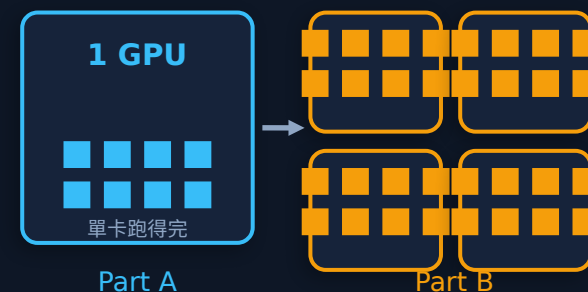


Transformer 上機： 從一張卡的每個 block，到多卡的 通訊

Part A：把玩具 Transformer 逐 block 攤開，看每一步跑在 GPU 的哪個單元、被算力還是頻寬卡住。

Part B：模型大到一張卡裝不下——資料平行有哪些問題，NVIDIA 用哪些跨 GPU 通訊新技術補上。



延續第一堂：roofline · 記憶體階層 · KV cache · decode 的 memory-bound

00 這堂課的兩條線

Part A

單卡逐 block × GPU

- 玩具 Transformer：T=6、d=6、2 heads（同構真實模型）
- 逐 block 拆：Embed → QKV → 多頭 → Attention → Add&Norm → FFN
- 每個 block 落在哪個單元：tensor core / CUDA core / HBM / L2 / shared
- 誰是 compute-bound、誰是 memory-bound
- 結論：整個模型一張卡（一顆 HBM）跑得完

Part B

多卡的資料平行與互連

- 一張卡裝不下：權重 + KV cache + optimizer 狀態
- 資料平行 (DP) 的四個問題（本堂重點）
- 切模型本身：張量 TP / 管線 PP / 專家 EP
- 通訊變瓶頸 → NVLink5 / NVSwitch / SHARP / NCCL
- 跨節點 InfiniBand / Spectrum-X；2026：Rubin、CMX

一條主線：先看懂一張卡怎麼跑一個 Transformer，再看模型變大後、卡與卡之間要搬什麼。

01 回顧：我們的玩具 Transformer

真實模型 d=12288、96 heads、128K context——大到畫不出來。把維度縮到極小、但保留每一個分叉，結構與尺寸無關。

玩具規格

維度	玩具	真實 (GPT 級)
序列長 T	6	128K+
d_model	6	12288
heads	2	96+
d_head	3	128
FFN hidden	12 (2×)	4×d

一層 Decoder block 的資料流（GPT 這類只留 decoder）



我愛喝咖啡 →

attention 是 token 間唯一交換資訊的地方；FFN 對每個 token 獨立（token 軸天生平行）。

同構：玩具上看懂的每一條資料流，乘上倍率就是真實模型。這堂課逐 block 問「它在 GPU 上跑在哪」。

02 我們要對應的那台 GPU

在逐 block 之前，先把 GPU 的零件擺出來——每個 block 會用到其中幾個。

運算單元（誰在算）

Tensor Core

整塊 tile 的矩陣乘加（MMA）→ 吃 GEMM

CUDA Core

純量逐格算 → softmax / norm / 逐元素

SM / warp

32 執行緒一組；大量 warp 藏記憶體延遲

記憶體階層（資料住哪）

HBM（global）

~3.35 TB/s · 大而慢 · 權重 + KV cache 住這

L2 cache

~ 數十 MB · 全晶片共用

Shared / L1

~228 KB/SM · 程式可控（tiling 關鍵）

Register

最快 · tensor core 直接吃

📄 互動環節① 切出去開 [interactive/gpu_map.html](https://interactive.gpu_map.html)：Cluster → Node → GPU → SM → 運算單元 (CUDA/Tensor) 下鑽，回來接逐 block。

03 Block 0 — Embedding + 位置編碼

0·Embed+PE

1·QKV 投影

2·多頭切分

3·Attention

4·Add & Norm

5·FFN

token id → 查表 (gather) → 向量 [6]**Embedding 表 [vocab × 6]**

住 HBM · 只讀取用到的幾列

+ 位置編碼：把「第幾個字」的資訊加進向量 → 得到 $X [6 \times 6]$ (逐元素相加)**→ 對應 NVIDIA GPU**

主要運算

查表 gather + 逐元素加

用到單元

CUDA core (不是
tensor core)

資料位置

embedding 表在 HBM

搬運型態

隨機讀取幾列 → 記憶體存
取**memory-bound** (運算極少)**第一步幾乎不算數：只是把字換成向量。GPU 這裡是在「搬」，不是在「算」。**

04 Block 1 — QKV 投影：第一個大矩陣乘

0·Embed+PE

1·QKV 投影

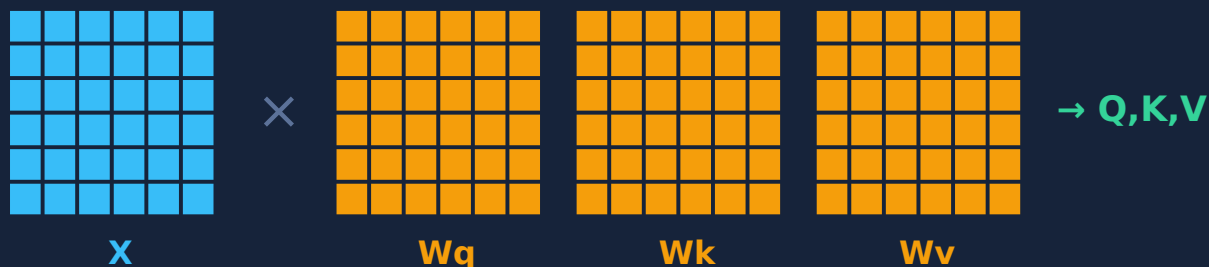
2·多頭切分

3·Attention

4·Add & Norm

5·FFN

$X [6 \times 6]$ 各乘三個權重 $\rightarrow Q, K, V [6 \times 6]$



這是標準 GEMM (矩陣 $\times$ 矩陣)。真實模型 $d=12288 \rightarrow$ 這幾個乘法是每層算力大戶。

$\rightarrow$ 對應 NVIDIA GPU

主要運算

GEMM : $X \cdot W$ (矩陣乘)

用到單元

Tensor Core (MMA 一次一塊 tile)

資料流動

HBM $\rightarrow$ **L2** $\rightarrow$ **shared** $\rightarrow$ **register**

加速關鍵

tiling : tile 留 shared 重複用

compute-bound (T 夠大時)

權重從 HBM 讀進來、切成 16×16 的 tile 餵進 tensor core——這就是 GPU 「算得快」的主場。

05 Block 2 — 多頭切分：分叉就是平行軸

0·Embed+PE

1·QKV 投影

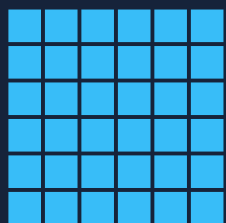
2·多頭切分

3·Attention

4·Add & Norm

5·FFN

6 維「切」成 2 個 head × 3 維——兩個 head 結構相同、彼此零依賴



Q,K,V [6×6]



head 1



head 2

→ 各自算 attention

多送 GPU 一個平行軸（head 軸）：head 之間可同時算，對到不同 tile / 不同 warp（batched GEMM）。

→ 對應 NVIDIA GPU

切出來的

head 軸（玩具 2、真實 96+）

平行對應

batched GEMM / 不同 warp、tile

為何重要

GPU 靠「寬」——平行軸越多越好餵

延伸

GQA：多個 Q head 共用一組 KV

餵飽平行度 → 高利用率

Transformer 天生「寬」：head 軸、token 軸都能平行——這正是它為 GPU 而生的原因（第一堂 Part 5）。

06 Block 3 — Attention：prefill 與 decode 兩張臉

0·Embed+PE

1·QKV 投影

2·多頭切分

3·Attention

4·Add & Norm

5·FFN

一個 head 的矩陣鏈： $Q \cdot K^T \rightarrow \text{softmax} \rightarrow \cdot V$

 $Q \cdot K^T$

GEMM/GEMV

softmax

CUDA core

 $\cdot V$

GEMM/GEMV

Prefill (吃整段)

$Q[6 \times 3] \cdot K^T = \text{GEMM}$ · 整批算 · 高 AI → compute-bound

Decode (逐字吐)

$q[1 \times 3] \cdot K^T = \text{GEMV}$ · KV cache 從 HBM 重讀 · AI ≈ 1 → memory-bound

→ 對應 NVIDIA GPU

 $Q \cdot K^T / \cdot V$

Tensor Core (矩陣乘)

softmax

CUDA core (逐元素 / 歸約)

KV cache

住 HBM (塞不進晶片)

decode 瓶頸

每步從 HBM 重串流 KV

prefill=compute / decode=memory

同一段 attention，prefill 吃算力、decode 吃頻寬——第一堂「<5% 利用率之謎」就發生在這個 block。

07 Block 4 — Add & LayerNorm：不上 tensor core 的那些

0·Embed+PE

1·QKV 投影

2·多頭切分

3·Attention

4·Add & Norm

5·FFN

殘差相加（Add）+ 層正規化（LayerNorm）

輸入 X

+

子層輸出

→

LayerNorm

這些是逐元素相加、求平均與變異數（歸約）——運算量很少，但要把整份 activation 從 HBM 讀進來、寫回去。

工程實務：常和前後運算「fuse（融合）」成一個 kernel，避免多跑一趟 HBM。

→ 對應 NVIDIA GPU

主要運算

逐元素加 + 歸約
（mean/var）

用到單元

CUDA core（非
tensor core）

資料流動

整份 activation 進出
HBM

優化手段

kernel fusion 減少
HBM 往返

memory-bound（算少、搬多）

Transformer 不是只有大矩陣乘。這些「小」運算是純 memory-bound，融合它們是省頻寬的日常功夫。

08 Block 5 — FFN：另一組大矩陣乘

0·Embed+PE

1·QKV 投影

2·多頭切分

3·Attention

4·Add & Norm

5·FFN

每個 token 獨立過兩層：6 → 12 → 6（中間放大 4×，玩具用 2×）



兩個 GEMM，中間夾一個非線性（GELU）。真實模型 FFN 常佔一層 2/3 的參數與算力。
token 之間零依賴 → 整批一起算。

→ 對應 NVIDIA GPU

主要運算

兩個 GEMM（W1、W2）

用到單元

Tensor Core 主場

GELU

CUDA core（逐元素）

平行軸

token 軸天然平行

compute-bound（大 GEMM）

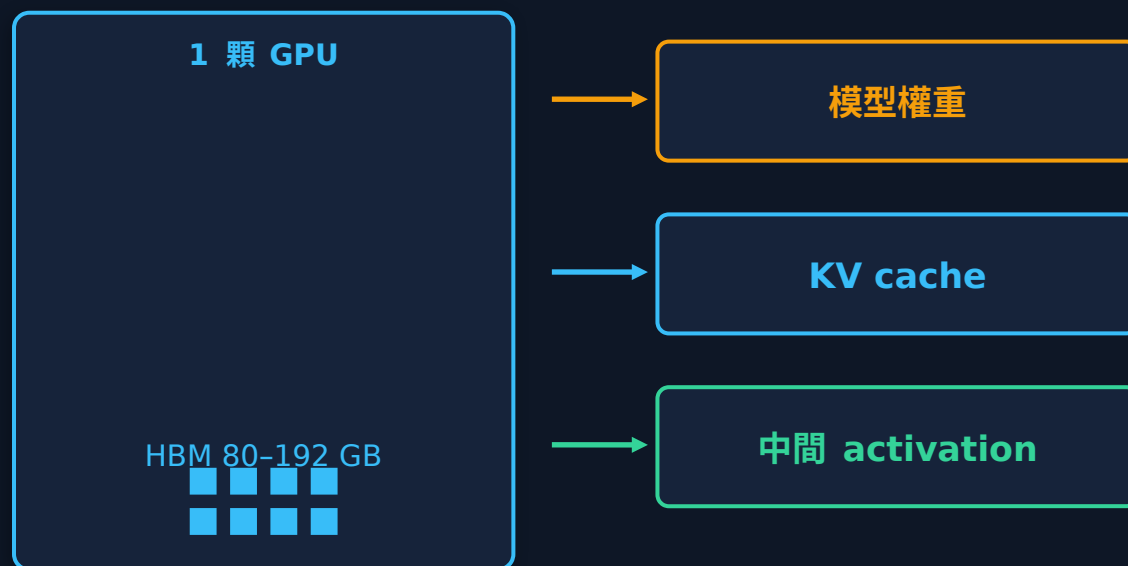
FFN 和 QKV 一樣是 tensor core 的主場。一層裡「大矩陣乘吃算力、其餘吃頻寬」的節奏就此定調。

09 一頁看完：block → GPU 單元 → 被誰卡住

#	Block	運算	GPU 單元	資料 / 搬運	判定
0	Embed + 位置	查表 gather + 加	CUDA core	HBM 讀幾列	memory
1	QKV 投影	GEMM	Tensor Core	tile 留 shared	compute
2	多頭切分	拆平行軸	warp / batched	—	結構
3	Attention	QK^T /softmax/ $\cdot V$	Tensor+CUDA	KV cache 住 HBM	pre=C / dec=M
4	Add & Norm	逐元素 + 歸約	CUDA core	activation 進出 HBM	memory
5	FFN	兩個 GEMM	Tensor Core	tile 留 shared	compute

☒ 互動環節② 開 [interactive/transformer_map.html](https://interactive.transformer-map.html)：全景 → Block → Attention → Head → 計算子 ($L2 \leftrightarrow HBM$) → FlashAttention(線上 softmax) → 硬體，切 訓練 /Prefill/Decode 看資料流。

10 Part A 收束：一張卡就跑得完



玩具 Transformer——甚至 7B 這種真實小模型——的權重、KV cache、activation 全部塞得進一顆 GPU 的 HBM。所有 block 的資料流都在同一顆晶片裡走完，沒有跨卡問題。

但是…… 70B、175B、甚至上兆參數的模型呢？一顆 HBM 放不下。這就是 Part B。

單卡的世界：看 **roofline**、看記憶體階層就夠了。跨過一顆 GPU 的容量，遊戲規則就變了。

11 為什麼一張卡不夠：三筆帳都在漲

以推論 / 訓練一個大模型為例，HBM 要同時裝下這幾樣——很快就爆掉 80-192 GB。

模型權重

70B fp16 $\approx$ 140 GB

175B $\approx$ 350 GB

一張 H100(80GB)/H200(141GB) 就放不下 70B 權重

KV cache

隨 序列長 $\times$ batch

$\times$ 層數 線性長大

長上下文 + 高併發 $\rightarrow$ 動輒數十 ~ 上百 GB

訓練狀態

梯度 + Adam 狀態

$\approx$ 參數量的 3-4 $\times$

訓練時遠比推論更吃容量（第一堂 Part 3）

模型放不進一顆 HBM $\rightarrow$ 必須拆到多張卡。問題是：拆什麼、卡跟卡之間要搬什麼？

12 資料平行 (Data Parallel)：最直覺的第一招

每張卡放「一份完整模型」，把 batch 切開各算各的，再把梯度對齊。



↓ 每步：梯度 all-reduce (把 4 張卡的梯度加總、再發回每張卡)

DP 擅長什麼

- 實作最簡單、最成熟
- 訓練吞吐幾乎線性擴充
- 推論可服務更多併發請求
- 模型「放得下一張卡」時的首選

資料平行 = 複製模型、切資料、同步梯度。切的是「資料」，不是「模型」——這正是它的天花板。

13 資料平行的四個問題（本堂重點）

① 模型沒有變小

每張卡仍要放整份模型 → 「一張卡裝不下」的模型，DP 根本救不了。

② 通訊隨規模長大

梯度 all-reduce 每步要搬 $\approx 2 \times$ 參數量；模型越大、卡越多，同步成本越兇。

③ 記憶體重複浪費

權重、梯度、optimizer 狀態每張卡各存一份 → ZeRO/FSDP 把它們切開，但換來更多通訊。

④ 推論延遲無解

decode 是 memory-bound；DP 只增吞吐、對單一請求的延遲沒幫助，KV cache 還各卡獨立。

①最致命：DP 切資料、不切模型。要裝下大模型、又想壓通訊，就得把「模型本身」拆開。

14 切模型本身：張量 / 管線 / 專家平行

既然 DP 不切模型，就換三種「切模型」的平行——各自切不同的軸，付不同的通訊代價。

Tensor Parallel

TP · 張量平行

切什麼

切「層內」：一個矩陣乘橫著切到多卡

每層 2 次 all-reduce

通訊最重 → 必須待在 NVLink 域

Pipeline Parallel

PP · 管線平行

切什麼

切「層間」：不同層放不同卡，像流水線

點對點傳 activation (p2p)

有 bubble (頭尾閒置)

Expert Parallel

EP · 專家平行

切什麼

切「MoE 專家」：不同 expert 放不同卡

all-to-all 路由 token

省 FLOPs、不省通訊

實務是「3D/4D 混合」：DP + TP + PP(+ EP) 疊用。切得越細、卡間通訊越密——通訊就成了新瓶頸。

15 通訊變瓶頸：頻寬階梯決定誰住哪

卡間搬資料的速度差一個數量級往下掉。哪種平行「通訊多」，就得住在「線快」的那一層。

晶片內 HBM

~3.35 TB/s

(單卡基準)

NVLink 5 / NVSwitch

~1.8 TB/s / GPU

scale-up：TP、EP 住這裡

跨節點 InfiniBand /
Spectrum-X

~50-100 GB/s / GPU

scale-out：DP、PP 可容忍

PCIe Gen5

~64 GB/s

沒 NVLink 時的卡間路（會被拖死）

心法：把「通訊最兇」的平行（TP/EP）綁在最快的線（NVLink）；「通訊稀」的（DP/PP）才放去跨節點。

16 NVIDIA 解法① scale-up：把多顆綁成一顆大 GPU

NVLink + NVSwitch 用「記憶體語意」把整個機架的 GPU 連成一個高頻寬域——TP/EP 的家。

NVLink 5	1.8 TB/s / GPU	18 條 × 100 GB/s；≈ PCIe Gen5 的 14×
NVSwitch（4 代）	全連通 fabric	72 顆非阻塞互連；可擴到 576 GPU / 1 PB/s
GB200 NVL72	72 顆 = 一個域	130 TB/s 聚合頻寬，當一顆「大 GPU」用

2026 最新 · Rubin

NVLink 6：3.6 TB/s / GPU（NVLink 5 的 2×）

Vera Rubin NVL72：260 TB/s 聚合

CES 2026 發表、H2 2026 出貨；NVL144/CPX 版把 prefill 拆出來做

scale-up 的意義：一個 NVLink 域內，72 顆 GPU 的 HBM 像一大池——大模型的 TP/EP 就攤在這池裡跑。

17 NVIDIA 解法② 把 all-reduce 搬進交換器：SHARP

傳統 all-reduce

每張卡互相收發、在 GPU 上做加總 → 佔 NVLink 頻寬、也佔 GPU 的 SM 去算加法。



互相收發、GPU 自己加

SHARP (in-network reduction)

加總直接在 NVSwitch / IB 交換器的晶片裡做完 → 省 NVLink 頻寬、也把 GPU 的 SM 解放出來算模型。



交換器算、發回結果

NCCL 是這一切的軟體層：all-reduce / all-gather / reduce-scatter / all-to-all 的實作，2.27 起 NVLink 與 IB 都能吃 SHARP。

梯度同步 (DP)、層內同步 (TP) 都靠 all-reduce——把它 offload 進交換器，通訊瓶頸就鬆一大截。

18 NVIDIA 解法③ scale-out：跨節點的網路

一個機架塞不下時，用網路把多台節點串成 AI factory——這裡走「訊息語意」（RDMA）。

InfiniBand (Quantum)

NDR 400 / XDR 800 Gb/s

HPC 老本行，原生支援 SHARP

Spectrum-X 乙太

為 AI 調過的以太網

adaptive routing + 壅塞控制，跑 RoCEv2

GPUDirect RDMA

遠端 NIC 直達 HBM

繞過 CPU，跨節點也能直接讀寫對方 GPU

scale-up vs scale-out

scale-up (NVLink)

記憶體語意、~TB/s、機架內 → 綁 TP/EP

scale-out (IB/乙太)

訊息語意、~百 Gb/s、跨節點 → 綁 DP/PP

兩層網路差一個數量級：所以「通訊密的平行留機架內、稀的才跨節點」不是選擇，是被頻寬逼的。

19 組起來：一個大模型怎麼跑在 NVL72 上

把前面的技術疊起來，看一個「一張卡裝不下」的模型實際怎麼被攤開。

① 一個 NVLink 域 = 一顆大 GPU

GB200 NVL72 把 72 顆連成 130 TB/s 的域 → 大模型的權重攤在這一池 HBM 裡放得下。

② 域內做 TP + EP

層內矩陣切到多卡、MoE 專家散到多卡，每層的 all-reduce / all-to-all 跑在 NVLink，並用 SHARP 加速。

③ 跨域做 DP + PP

多個 NVL72 之間用 InfiniBand / Spectrum-X 做資料平行的梯度同步、管線的 activation 傳遞。

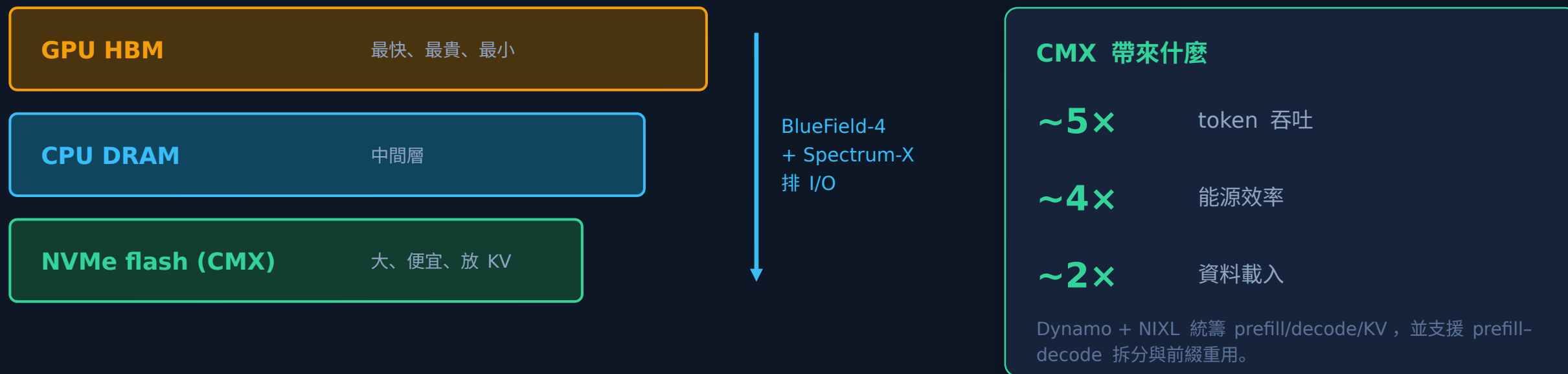
④ NCCL 統籌

上層一律用 NCCL 呼叫集合通訊，自動選 NVLink 還是網路、要不要走 SHARP。

真實訓練 = DP × TP × PP × EP 的混合，每一軸配一種通訊、一條線——互連技術就是讓這套疊得起來。

20 2026 新招 CMX：把 KV cache 搬出 HBM

呼應第一堂：decode 是 memory-bound，KV cache 又吃 HBM 容量。CMX 給它一層專屬儲存。



互連技術不只服務訓練：把 KV cache 卸到網路掛載的 flash，直接打 decode 的頻寬 / 容量牆。

21 互動環節③：多卡平行地圖

切出去開 [interactive/parallelism_map.html](#) ——把同一個玩具 Transformer 攤到多張卡上。

1 單卡

整份模型在一顆 HBM → 沒有通訊

2 裝不下

模型長大 → 權重 /KV/ 狀態溢出一顆卡

3 資料平行 DP

複製模型、切 batch、梯度 all-reduce (看四個問題)

4 張量平行 TP

把「一層」的 $W_q/W_k/W_v/W_o + W_1/W_2$ 每個矩陣切 4 片、每層 all-reduce

5 管線平行 PP

整層分段 (不切一層內部)、p2p 傳 activation、看 bubble

6 互連硬體

各平行走 NVLink/NVSwitch 還是 IB/Spectrum-X (scale-up vs scale-out)

操作：數字鍵 1-6 跳層 · Esc 回上層 (TP / PP 兩層會畫出「一層」的權重矩陣怎麼被切)

抽象的「切哪個軸、走哪條線」變成看得到、玩得到的東西——這是本堂的落地。

22 帶走三句話

1

一張卡：逐 block 看單元

Transformer 每個 block 落在 tensor core（大 GEMM）或 CUDA core（softmax/norm/ 查表）；大矩陣吃算力、其餘吃頻寬。

2

多卡：DP 切資料、不切模型

資料平行救不了「裝不下」的模型，還有通訊 / 重複 / 延遲四問題 → 得靠 TP/PP/EP 切模型本身。

3

互連決定切法的可行性

通訊階梯逼你把 TP/EP 綁 NVLink 域、DP/PP 才跨節點；NVSwitch+SHARP、Rubin/NVLink6、CMX 都在鬆這道牆。

下一步：把 demo 在多卡 GPU 上實跑，量 all-reduce 頻寬與 TP 的通訊佔比。